

**TUGAS BESAR PERGUDANGAN DATA
KELOMPOK 18 RA
MISI 2**

Nama Anggota Kelompok :

1. Keren Marito (123450020)
2. Aditya Taufiqurrohman (123450032)
3. Lia Hana Ichisasmita (123450089)



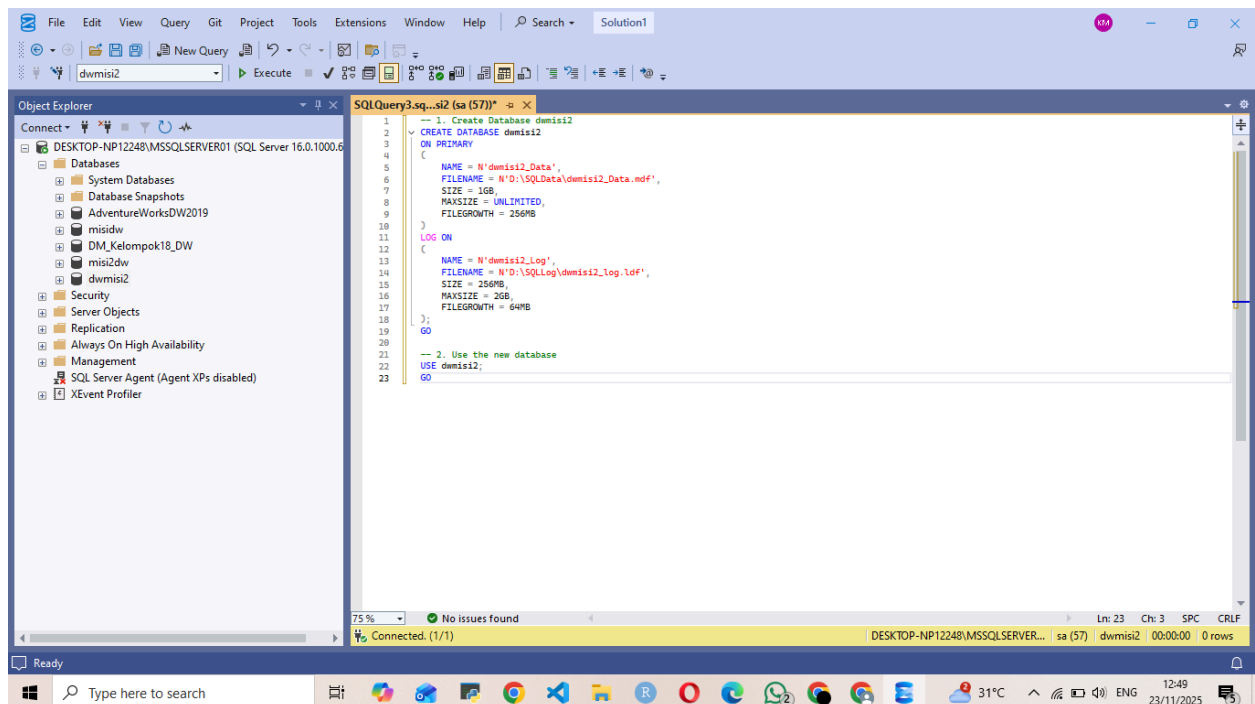
**PROGRAM STUDI SAINS DATA
FAKULTAS SAINS
INSTITUT TEKNOLOGI SUMATERA
2025**

Step 1:

1. Database Setup

```
-- 1. Create Database dwmisi2
CREATE DATABASE dwmisi2
ON PRIMARY
(
    NAME = N'dwmisi2_Data',
    FILENAME = N'D:\SQLData\dwmisi2_Data.mdf',
    SIZE = 1GB,
    MAXSIZE = UNLIMITED,
    FILEGROWTH = 256MB
)
LOG ON
(
    NAME = N'dwmisi2_Log',
    FILENAME = N'D:\SQLLog\dwmisi2_log.ldf',
    SIZE = 256MB,
    MAXSIZE = 2GB,
    FILEGROWTH = 64MB
);
GO

-- 2. Use the new database
USE dwmisi2;
GO
```



2. Create Dimension Tables

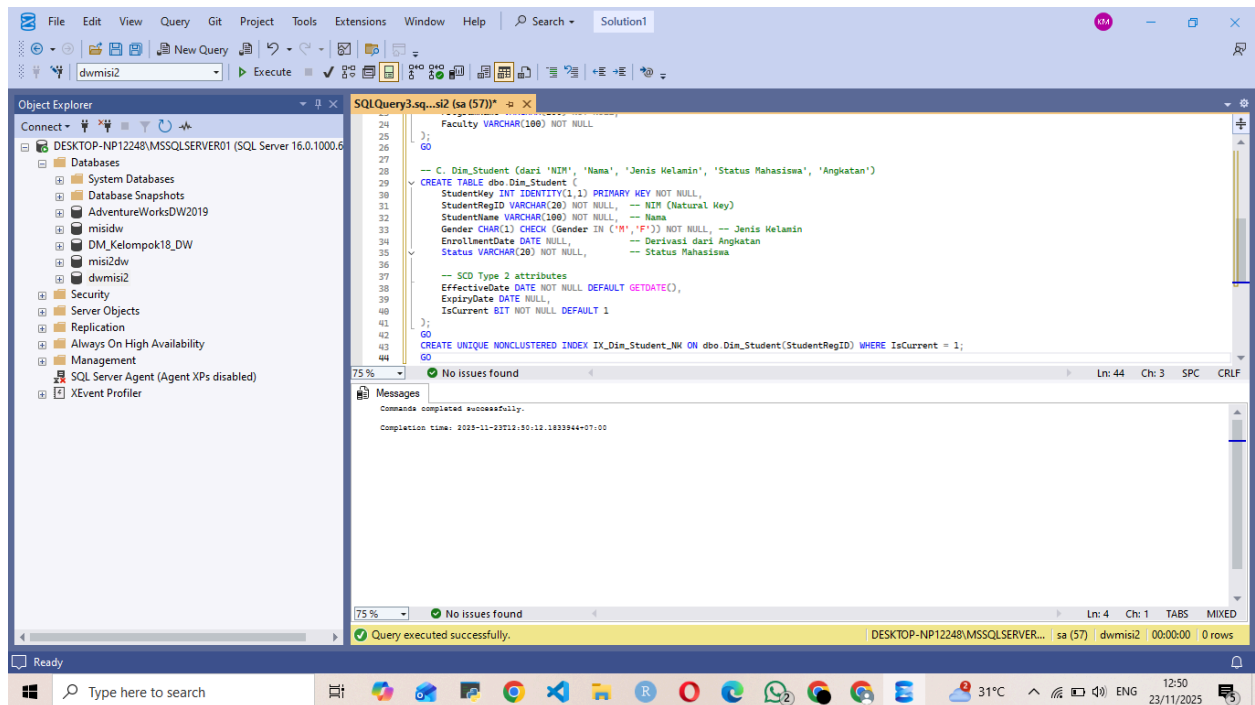
```
USE dwmisi2;
GO

-- A. Dim_Date
CREATE TABLE dbo.Dim_Date (
    DateKey INT PRIMARY KEY NOT NULL,
    FullDate DATE NOT NULL,
    DayNumberOfWeek TINYINT NOT NULL,
    DayName VARCHAR(10) NOT NULL,
    MonthNumber TINYINT NOT NULL,
    MonthName VARCHAR(10) NOT NULL,
    Year SMALLINT NOT NULL,
    AcademicYear VARCHAR(9) NULL,
    Semester TINYINT NULL
);
GO
ALTER TABLE dbo.Dim_Date ADD CONSTRAINT CK_Dim_Date_Month CHECK
(MonthNumber BETWEEN 1 AND 12);
GO

-- B. Dim_Program (dari 'Program Studi' dan 'Fakultas')
CREATE TABLE dbo.Dim_Program (
    ProgramKey INT IDENTITY(1,1) PRIMARY KEY NOT NULL,
    ProgramName VARCHAR(100) NOT NULL,
    Faculty VARCHAR(100) NOT NULL
);
GO

-- C. Dim_Student (dari 'NIM', 'Nama', 'Jenis Kelamin', 'Status Mahasiswa', 'Angkatan')
CREATE TABLE dbo.Dim_Student (
    StudentKey INT IDENTITY(1,1) PRIMARY KEY NOT NULL,
    StudentRegID VARCHAR(20) NOT NULL, -- NIM (Natural Key)
    StudentName VARCHAR(100) NOT NULL, -- Nama
    Gender CHAR(1) CHECK (Gender IN ('M','F')) NOT NULL, -- Jenis Kelamin
    EnrollmentDate DATE NULL, -- Derivasi dari Angkatan
    Status VARCHAR(20) NOT NULL, -- Status Mahasiswa

    -- SCD Type 2 attributes
    EffectiveDate DATE NOT NULL DEFAULT GETDATE(),
    ExpiryDate DATE NULL,
    IsCurrent BIT NOT NULL DEFAULT 1
);
GO
CREATE UNIQUE NONCLUSTERED INDEX IX_Dim_Student_NK ON
dbo.Dim_Student(StudentRegID) WHERE IsCurrent = 1;
GO
```



3. Create Fact Tables

```
USE dwmisi2;
GO
```

```
CREATE TABLE dbo.Fact_Enrollment (
    EnrollmentKey BIGINT IDENTITY(1,1) NOT NULL,
```

```
-- Foreign Keys
```

```
DateKey INT NOT NULL,
StudentKey INT NOT NULL,
ProgramKey INT NOT NULL,
```

```
-- Degenerate Dimensions & Measures
```

```
Semester TINYINT NOT NULL,
Nilai DECIMAL(3,1) NOT NULL,
IPK DECIMAL(3,2) NOT NULL,
```

```
-- Metadata
```

```
SourceSystem VARCHAR(50) NOT NULL DEFAULT 'Data_Mahasiswa',
LoadDate DATETIME DEFAULT GETDATE(),
```

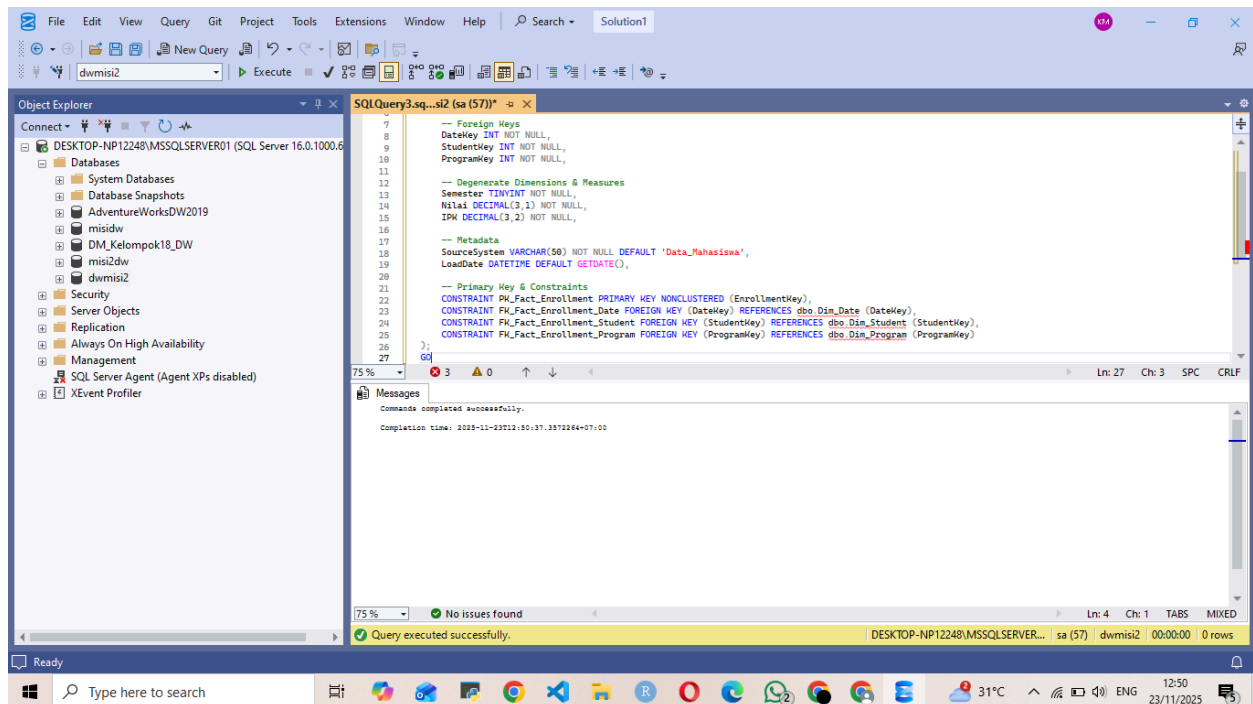
```
-- Primary Key & Constraints
```

```
CONSTRAINT PK_Fact_Enrollment PRIMARY KEY NONCLUSTERED (EnrollmentKey),
CONSTRAINT FK_Fact_Enrollment_Date FOREIGN KEY (DateKey) REFERENCES
dbo.Dim_Date (DateKey),
```

```

        CONSTRAINT FK_Fact_Enrollment_Student FOREIGN KEY (StudentKey) REFERENCES
        dbo.Dim_Student (StudentKey),
        CONSTRAINT FK_Fact_Enrollment_Program FOREIGN KEY (ProgramKey)
        REFERENCES dbo.Dim_Program (ProgramKey)
    );
GO

```



Step 2:

4. Indexing Strategy

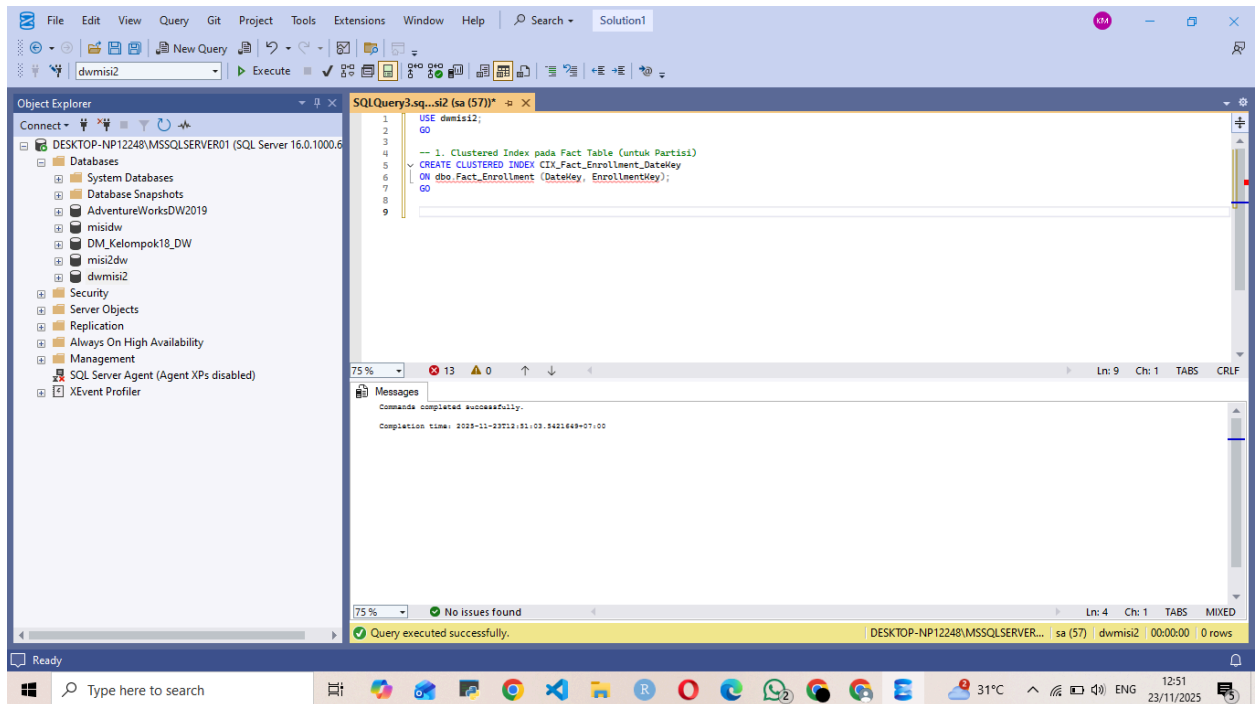
a. Clustered Index on Fact Table

```

USE dwmisi2;
GO

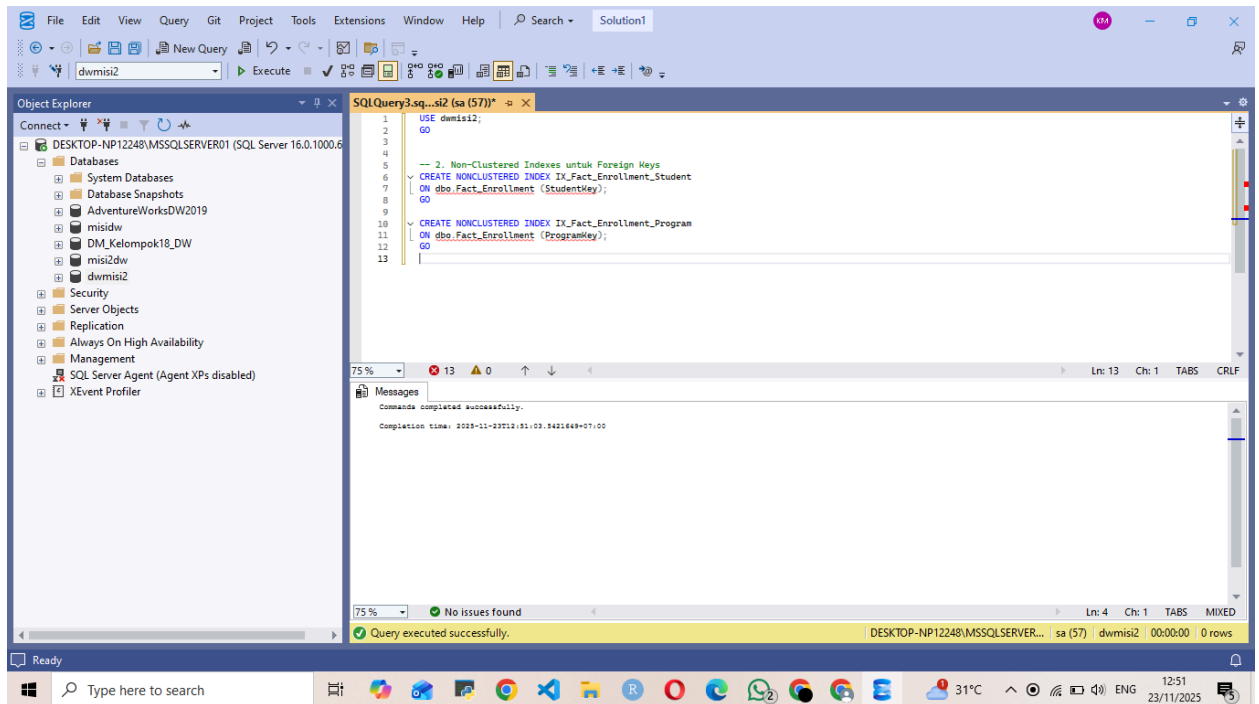
-- 1. Clustered Index pada Fact Table (untuk Partisi)
CREATE CLUSTERED INDEX CIX_Fact_Enrollment_DateKey
ON dbo.Fact_Enrollment (DateKey, EnrollmentKey);
GO

```



b. Non-Clustered Indexes

```
USE dwmisi2;  
GO  
  
-- 2. Non-Clustered Indexes untuk Foreign Keys  
CREATE NONCLUSTERED INDEX IX_Fact_Enrollment_Student  
ON dbo.Fact_Enrollment (StudentKey);  
GO  
  
CREATE NONCLUSTERED INDEX IX_Fact_Enrollment_Program  
ON dbo.Fact_Enrollment (ProgramKey);  
GO
```



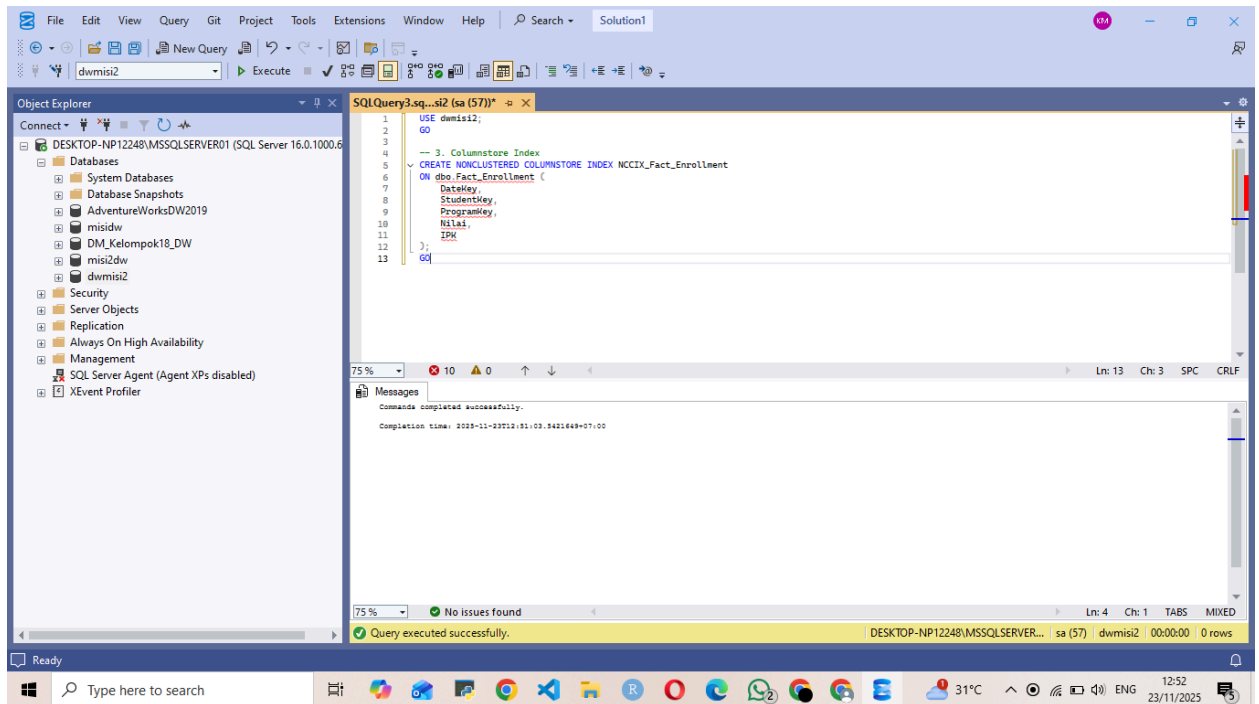
c. Columnstore Index (for large fact tables)

```

USE dwmisi2;
GO

-- 3. Columnstore Index
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCIX_Fact_Enrollment
ON dbo.Fact_Enrollment (
    DateKey,
    StudentKey,
    ProgramKey,
    Nilai,
    IPK
);
GO

```



Step 3:

5. Partitioning Strategy

```
USE dwmisi2;
GO
```

```
-- 1. Create Partition Function (PF_AcademicYear)
CREATE PARTITION FUNCTION PF_AcademicYear (INT)
AS RANGE RIGHT FOR VALUES
```

```
(
    20200801, -- Batas Akademik Year
    20210801,
    20220801,
    20230801,
    20240801,
    20250801
);
GO
```

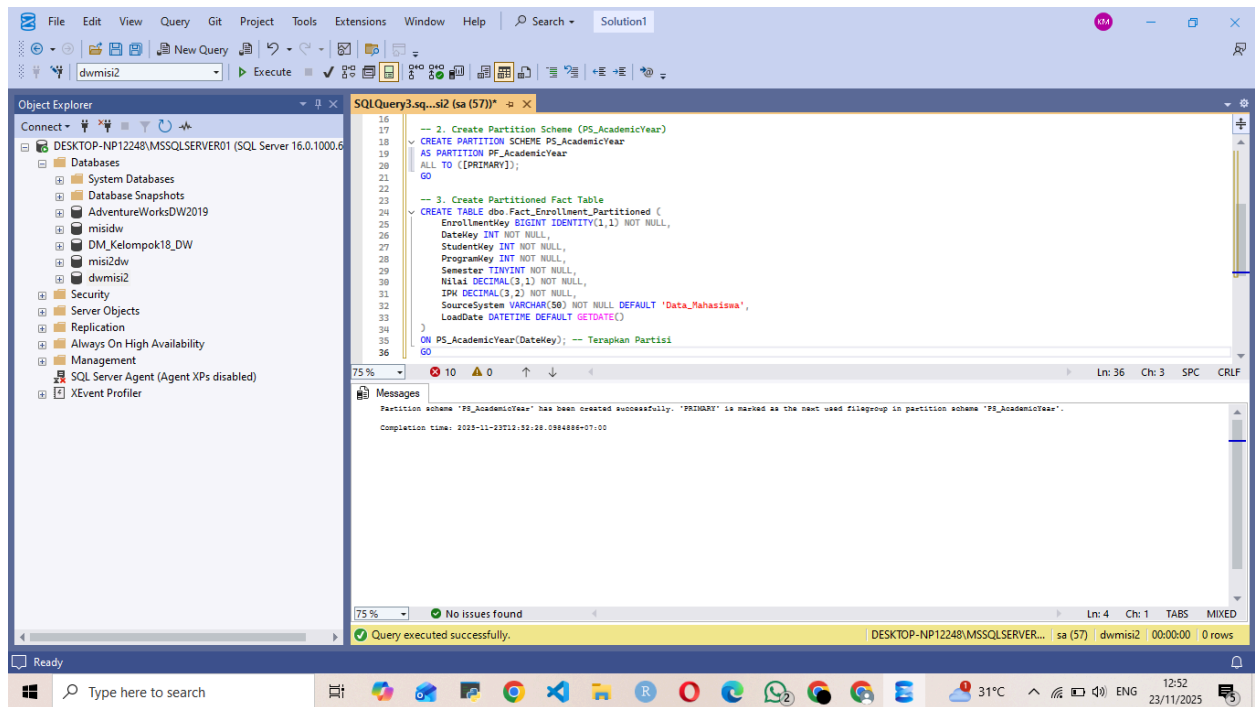
```
-- 2. Create Partition Scheme (PS_AcademicYear)
CREATE PARTITION SCHEME PS_AcademicYear
AS PARTITION PF_AcademicYear
ALL TO ([PRIMARY]);
GO
```

```
-- 3. Create Partitioned Fact Table
```

```

CREATE TABLE dbo.Fact_Enrollment_Partitioned (
    EnrollmentKey BIGINT IDENTITY(1,1) NOT NULL,
    DateKey INT NOT NULL,
    StudentKey INT NOT NULL,
    ProgramKey INT NOT NULL,
    Semester TINYINT NOT NULL,
    Nilai DECIMAL(3,1) NOT NULL,
    IPK DECIMAL(3,2) NOT NULL,
    SourceSystem VARCHAR(50) NOT NULL DEFAULT 'Data_Mahasiswa',
    LoadDate DATETIME DEFAULT GETDATE()
)
ON PS_AcademicYear(DateKey); -- Terapkan Partisi
GO

```



Step 4:

6. ETL Architecture Design

- Source to Staging: Extract raw data dari Data_Mahasiswa.xlsx
- Staging to Integration: Data cleansing (trim, upper, normalisasi gender, validasi tanggal)
- Integration ke Data Warehouse: Transformasi dan load ke Dim_Student, Dim_Program, Fact_Enrollment.

7. Create Staging Tables

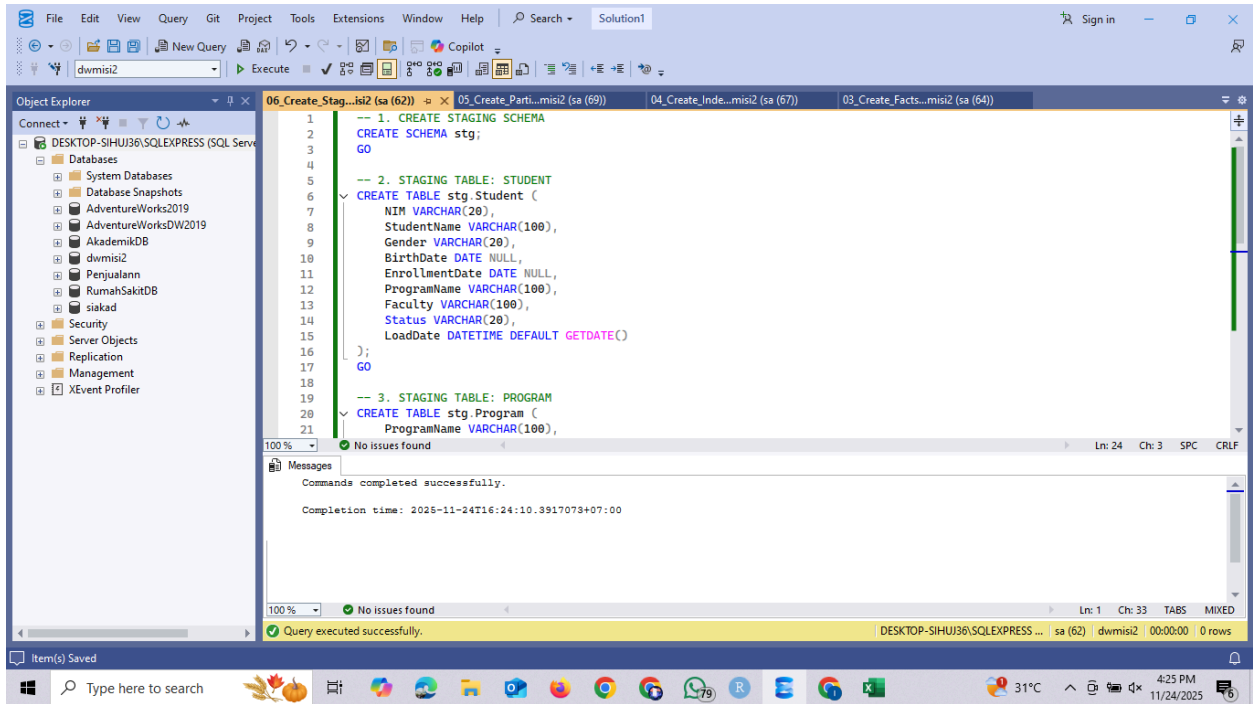
```
-- 1. CREATE STAGING SCHEMA
```

```
CREATE SCHEMA stg;
GO

-- 2. STAGING TABLE: STUDENT
CREATE TABLE stg.Student (
    NIM VARCHAR(20),
    StudentName VARCHAR(100),
    Gender VARCHAR(20),
    BirthDate DATE NULL,
    EnrollmentDate DATE NULL,
    ProgramName VARCHAR(100),
    Faculty VARCHAR(100),
    Status VARCHAR(20),
    LoadDate DATETIME DEFAULT GETDATE()
);
GO

-- 3. STAGING TABLE: PROGRAM
CREATE TABLE stg.Program (
    ProgramName VARCHAR(100),
    Faculty VARCHAR(100),
    LoadDate DATETIME DEFAULT GETDATE()
);
GO

-- 4. STAGING TABLE: ENROLLMENT (NILAI)
CREATE TABLE stg.Enrollment (
    EnrollmentID VARCHAR(50),
    NIM VARCHAR(20),
    EnrollmentDate DATE NULL,
    ProgramName VARCHAR(100),
    Semester VARCHAR(10),
    AcademicYear VARCHAR(9),
    Grade DECIMAL(3,1),
    IPK DECIMAL(3,2),
    LoadDate DATETIME DEFAULT GETDATE()
);
GO
```



8. ETL Mapping Document

Source Table	Source Column	Target Table	Target Column	Transformation
mahasiswa	nim	Dim_Student	StudentRegID	Direct Mapping
mahasiswa	nama	Dim_Student	StudentName	UPPER(TRIM(nama))
mahasiswa	jenis_kelamin	Dim_Student	Gender	CASE WHEN jenis_kelamin='L' THEN 'M' ELSE 'F' END
mahasiswa	angkatan	Dim_Student	EnrollmentDate	CAST(angkatan+'-09-01' AS DATE)
prodi	ProgramName	Dim_Program	ProgramName	TRIM(ProgramName)
nilai	nilai	Fact_Enrollment	Nilai	CONVERT(NUMERIC(3,1), nilai)
nilai	ipk	Fact_Enrollment	IPK	CONVERT(NUMERIC(3,2), ipk)

Step 5:

9. Create SSIS Project Menggunakan T-SQL Stored Procedures

CREATE PROCEDURE dbo.usp_Load_Dim_Program

```

AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO dbo.Dim_Program (
        ProgramName,
        Faculty
    )
    SELECT DISTINCT
        TRIM(p.ProgramName),
        TRIM(p.Faculty)
    FROM stg.Program p
    WHERE NOT EXISTS (
        SELECT 1 FROM dbo.Dim_Program dp
        WHERE TRIM(dp.ProgramName) = TRIM(p.ProgramName)
        AND TRIM(dp.Faculty) = TRIM(p.Faculty)
    );
END;
GO

```

```

CREATE PROCEDURE dbo.usp_Load_Dim_Student
AS
BEGIN
    SET NOCOUNT ON;
    -- Expire old records (SCD2)
    UPDATE d
    SET ExpiryDate = GETDATE(), IsCurrent = 0
    FROM dbo.Dim_Student d
    INNER JOIN stg.Student s ON d.StudentRegID = s.NIM
    WHERE d.IsCurrent = 1
    AND (
        d.StudentName <> UPPER(TRIM(s.StudentName))
        OR d.Gender <> CASE WHEN s.Gender = 'L' THEN 'M' ELSE 'F' END
        OR d.Status <> s.Status
    );
    -- Insert new/changed records
    INSERT INTO dbo.Dim_Student (
        StudentRegID,
        StudentName,
        Gender,
        EnrollmentDate,
        Status,
        EffectiveDate,
        IsCurrent
    )
    SELECT
        s.NIM,
        UPPER(TRIM(s.StudentName)),
        CASE WHEN s.Gender = 'L' THEN 'M' ELSE 'F' END,
        CAST(s.EnrollmentDate AS DATE),
        s.Status,

```

```

        GETDATE(),
        1
    FROM stg.Student s
    WHERE NOT EXISTS (
        SELECT 1 FROM dbo.Dim_Student d
        WHERE s.NIM = d.StudentRegID AND d.IsCurrent = 1
    );
END;
GO

CREATE PROCEDURE dbo.usp_Load_Fact_Enrollment
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO dbo.Fact_Enrollment (
        DateKey,
        StudentKey,
        ProgramKey,
        Semester,
        Nilai,
        IPK,
        SourceSystem,
        LoadDate
    )
    SELECT
        CAST(CONVERT(VARCHAR(8), e.EnrollmentDate, 112) AS INT) AS DateKey,
        ds.StudentKey,
        dp.ProgramKey,
        CONVERT(TINYINT, e.Semester),
        CONVERT(DECIMAL(3,1), e.Grade),
        CONVERT(DECIMAL(3,2), e.IPK),
        'Data_Mahasiswa',
        GETDATE()
    FROM stg.Enrollment e
    INNER JOIN dbo.Dim_Student ds ON e.NIM = ds.StudentRegID AND ds.IsCurrent = 1
    INNER JOIN dbo.Dim_Program dp ON e.ProgramName = dp.ProgramName
    WHERE NOT EXISTS (
        SELECT 1 FROM dbo.Fact_Enrollment fe
        WHERE fe.StudentKey = ds.StudentKey
        AND fe.ProgramKey = dp.ProgramKey
        AND fe.DateKey = CAST(CONVERT(VARCHAR(8), e.EnrollmentDate, 112) AS INT)
        AND fe.Semester = CONVERT(TINYINT, e.Semester)
    );
END;
GO

CREATE PROCEDURE dbo.usp_Master_ETL
AS
BEGIN
    DECLARE @ErrorMessage NVARCHAR(4000);

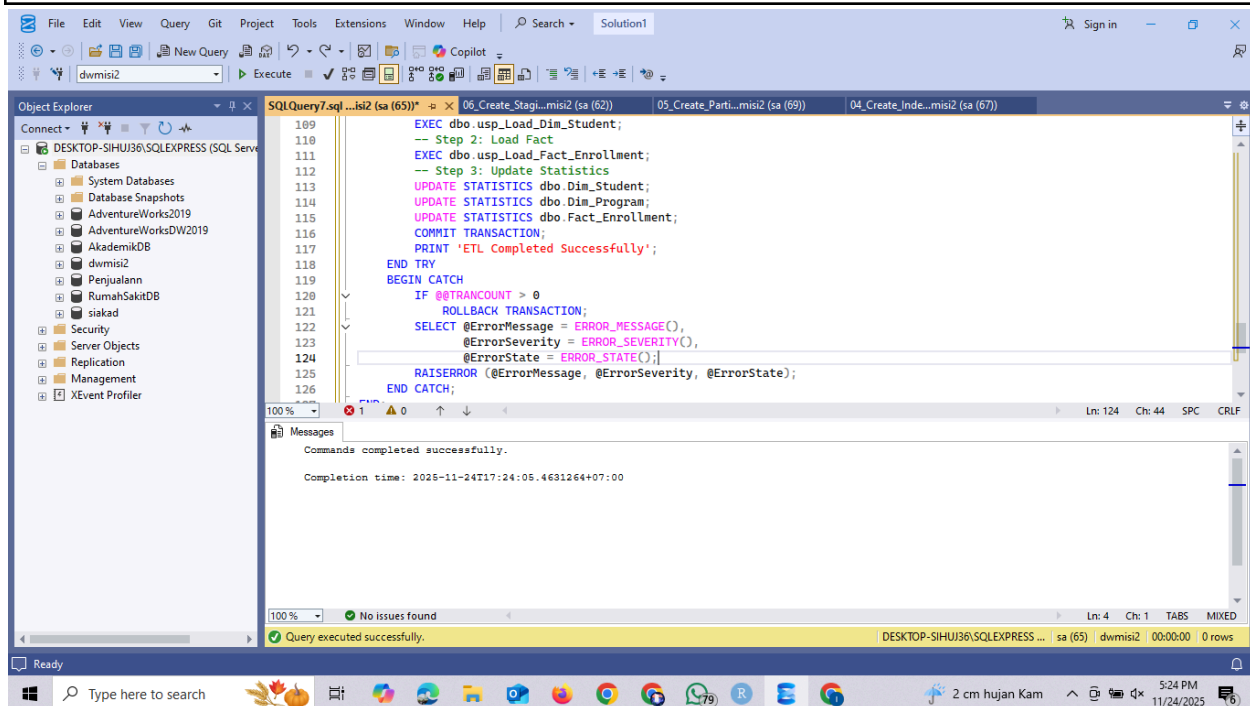
```

```

DECLARE @ErrorSeverity INT;
DECLARE @ErrorState INT;

BEGIN TRY
    BEGIN TRANSACTION;
    -- Step 1: Load Dimensions
    EXEC dbo.usp_Load_Dim_Program;
    EXEC dbo.usp_Load_Dim_Student;
    -- Step 2: Load Fact
    EXEC dbo.usp_Load_Fact_Enrollment;
    -- Step 3: Update Statistics
    UPDATE STATISTICS dbo.Dim_Student;
    UPDATE STATISTICS dbo.Dim_Program;
    UPDATE STATISTICS dbo.Fact_Enrollment;
    COMMIT TRANSACTION;
    PRINT 'ETL Completed Successfully';
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    SELECT @ErrorMessage = ERROR_MESSAGE(),
           @ErrorSeverity = ERROR_SEVERITY(),
           @ErrorState = ERROR_STATE();
    RAISERROR (@ErrorMessage, @ErrorSeverity, @ErrorState);
END CATCH;
END;
GO

```



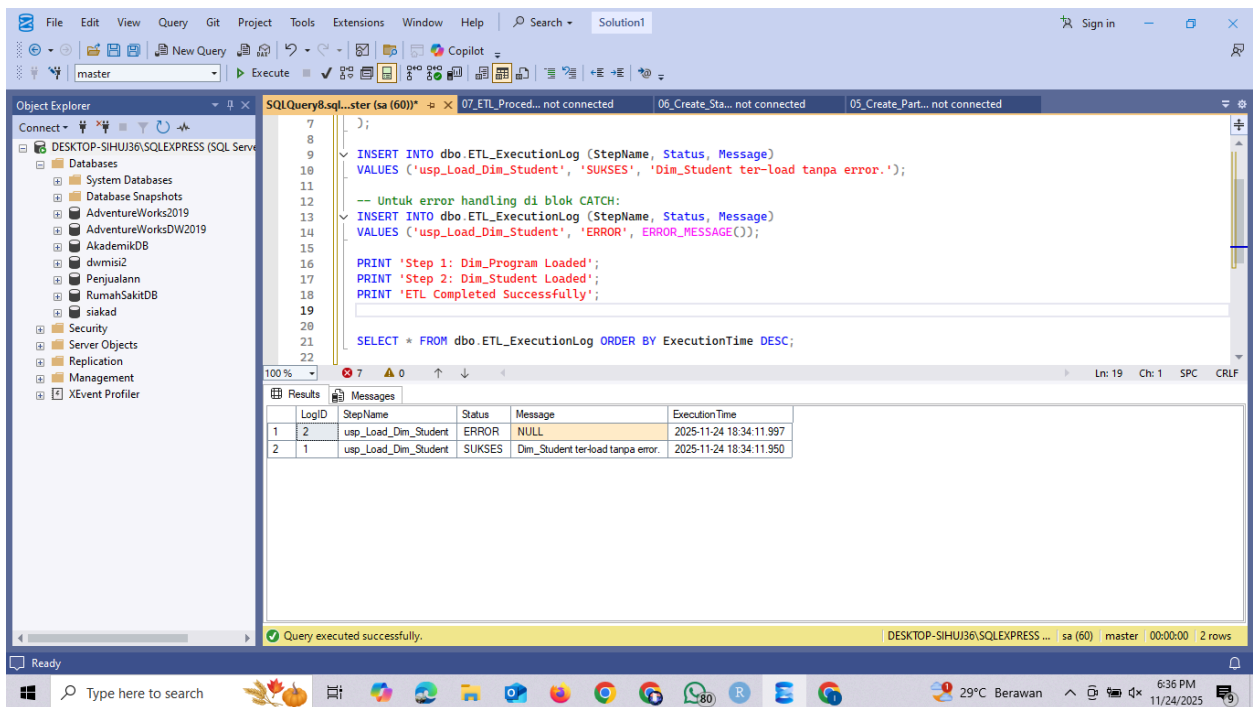
10. ETL Execution Log

```
CREATE TABLE dbo.ETL_ExecutionLog (  
    LogID INT IDENTITY(1,1) PRIMARY KEY,  
    StepName NVARCHAR(100),  
    Status NVARCHAR(20),  
    Message NVARCHAR(4000),  
    ExecutionTime DATETIME DEFAULT GETDATE()  
);  
  
INSERT INTO dbo.ETL_ExecutionLog (StepName, Status, Message)  
VALUES ('usp_Load_Dim_Student', 'SUKSES', 'Dim_Student ter-load tanpa error.');
```

-- Untuk error handling di blok CATCH:
INSERT INTO dbo.ETL_ExecutionLog (StepName, Status, Message)
VALUES ('usp_Load_Dim_Student', 'ERROR', ERROR_MESSAGE());

```
PRINT 'Step 1: Dim_Program Loaded';  
PRINT 'Step 2: Dim_Student Loaded';  
PRINT 'ETL Completed Successfully';
```

```
SELECT * FROM dbo.ETL_ExecutionLog ORDER BY ExecutionTime DESC;
```



The screenshot shows the SQL Server Enterprise Manager interface. The query window displays the following SQL code:

```
7 );  
8  
9  
10 INSERT INTO dbo.ETL_ExecutionLog (StepName, Status, Message)  
11 VALUES ('usp_Load_Dim_Student', 'SUKSES', 'Dim_Student ter-load tanpa error.');
```

-- Untuk error handling di blok CATCH:
INSERT INTO dbo.ETL_ExecutionLog (StepName, Status, Message)
VALUES ('usp_Load_Dim_Student', 'ERROR', ERROR_MESSAGE());

```
15  
16 PRINT 'Step 1: Dim_Program Loaded';  
17 PRINT 'Step 2: Dim_Student Loaded';  
18 PRINT 'ETL Completed Successfully';  
19  
20  
21 SELECT * FROM dbo.ETL_ExecutionLog ORDER BY ExecutionTime DESC;  
22
```

The Results pane shows the following data:

LogID	StepName	Status	Message	ExecutionTime
1	usp_Load_Dim_Student	ERROR	NULL	2025-11-24 18:34:11.997
2	usp_Load_Dim_Student	SUKSES	Dim_Student ter-load tanpa error.	2025-11-24 18:34:11.950

The Messages pane shows the following error message:

```
Msg 50000, Level 16, State 1, Procedure usp_Load_Dim_Student, Line 15, Batch 1  
An error has occurred. See the error message in the message box.
```

The status bar at the bottom indicates: Query executed successfully. DESKTOP-SIHUI36\SQLEXPRESS ... sa (60) master 00:00:00 2 rows

Tabel Excuton Log

LogID	StepName	Status	Message	Execution Time
1	usp_Load_Dim_Student	ERROR	NULL	2025-11-24 18:34:11.997
2	usp_Load_Dim_Student	SUKSES	Dim_Student ter-load tanpa error.	2025-11-24 18:34:11.950

Step 6: Data Quality Assurance

11. Data Quality Checks

```

USE dwmisi2;
GO

-- 1. Check Completeness: NULL Values pada Dim_Student
SELECT
    'Completeness: NULL Values' AS [Check],
    COUNT(*) AS TotalRows,
    SUM(CASE WHEN StudentName IS NULL THEN 1 ELSE 0 END) AS Null_StudentName,
    SUM(CASE WHEN BirthDate IS NULL THEN 1 ELSE 0 END) AS Null_BirthDate,
    SUM(CASE WHEN Gender IS NULL THEN 1 ELSE 0 END) AS Null_Gender
FROM dbo.Dim_Student;
GO

-- 2. Check Consistency: Orphan Fact Enrollment (fact tanpa master)
SELECT
    'Consistency: Orphan Fact_Enrollment' AS [Check],
    COUNT(*) AS OrphanRecordCount
FROM dbo.Fact_Enrollment f
LEFT JOIN dbo.Dim_Student s ON f.StudentKey = s.StudentKey
WHERE s.StudentKey IS NULL;
GO

-- 3. Check Accuracy: Valid range Grade/IPK pada Fact_Enrollment
SELECT
    'Accuracy: Invalid Grade Range' AS [Check],
    COUNT(*) AS InvalidGradeCount
FROM dbo.Fact_Enrollment
WHERE Grade < 0.00 OR Grade > 4.00;
GO

-- 4. Check Duplicates pada Dim_Student
SELECT
    'DuplicateCheck: StudentRegID' AS [Check],
    StudentRegID,
    COUNT(*) AS DuplicateCount
FROM dbo.Dim_Student
GROUP BY StudentRegID
HAVING COUNT(*) > 1;

```

GO

-- 5. Record Count Reconciliation

SELECT

'RecordCount: Dim_Student' AS [Check],
COUNT(*) AS RecordCount

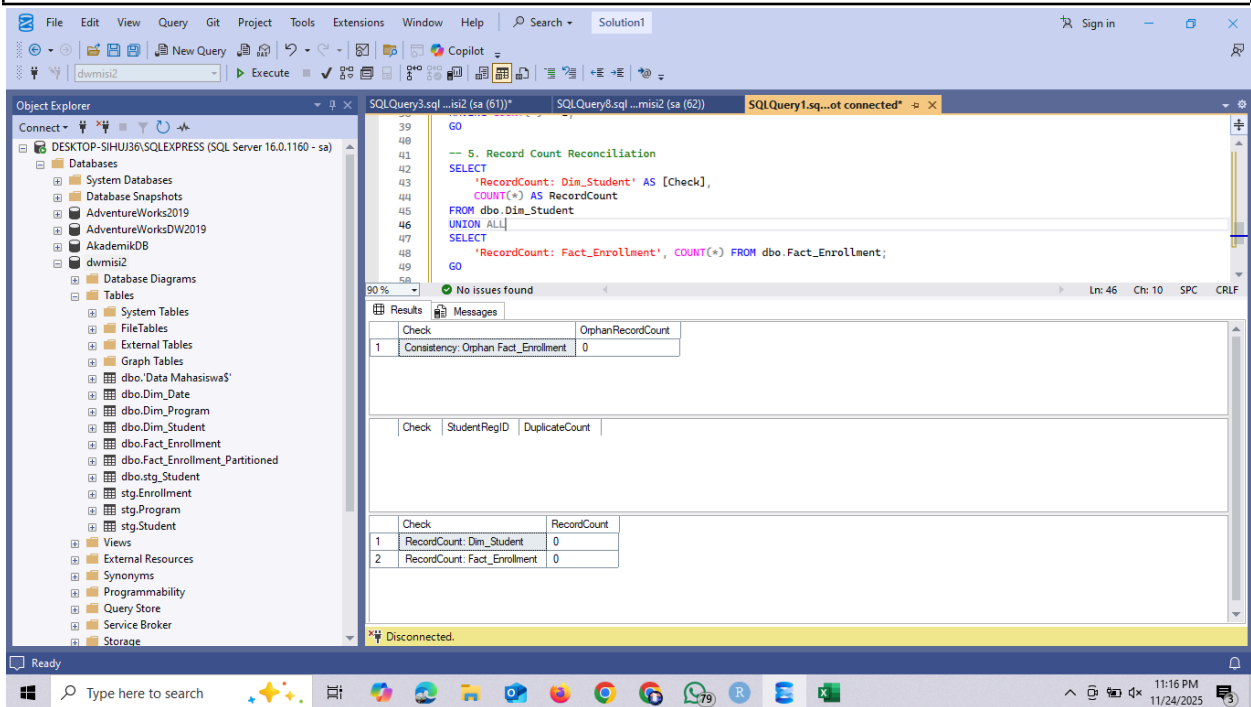
FROM dbo.Dim_Student

UNION ALL

SELECT

'RecordCount: Fact_Enrollment', COUNT(*) FROM dbo.Fact_Enrollment;

GO



Step 7: Performance Testing

13. Create Test Queries

-- Query 1: Total students per program

SET STATISTICS TIME ON;

SET STATISTICS IO ON;

SELECT

p.ProgramName,
COUNT(DISTINCT f.StudentKey) AS TotalStudents,
SUM(f.Credit) AS TotalCredits

FROM

```

Fact_Enrollment f
INNER JOIN Dim_Program p ON f.ProgramKey = p.ProgramKey
INNER JOIN Dim_Date d ON f.DateKey = d.DateKey
WHERE
    d.AcademicYear = '2024/2025'
GROUP BY
    p.ProgramName
ORDER BY
    TotalStudents DESC;

-- Query 2: Monthly enrollments trend
SELECT
    d.MonthNumber,
    d.MonthName,
    COUNT(DISTINCT f.EnrollmentKey) AS TotalEnrollments,
    COUNT(f.StudentKey) AS TotalStudents
FROM
    Fact_Enrollment f
INNER JOIN Dim_Date d ON f.DateKey = d.DateKey
GROUP BY
    d.Year, d.MonthNumber, d.MonthName
ORDER BY
    d.Year, d.MonthNumber, d.MonthName;

```

